

List and Array operations

List to Array

Description: Converts a JavaScript array into a linked list structure by iterating backwards through the array and building pairs.

Parameters: arr: Array<any> - The array to convert

Returns: List - A linked list representation of the array

Example: `array_to_list([1, 2, 3]);`
// Returns: pair(1, pair(2, pair(3, null)))

```
1 function array_to_list(arr) {
2   let result = null;
3   for (let i = array_length(arr) - 1; i >= 0; i = i - 1) {
4     result = pair(arr[i], result);
5   }
6   return result;
7 }
```

Array to List

Description: Converts a linked list into a JavaScript array by recursively traversing the list and collecting elements.

Parameters: lst: List - The linked list to convert

Returns: Array<any> - An array containing all elements from the list

Example: `list_to_array(list(1, 2, 3));`
// Returns: [1, 2, 3]

```
1 function list_to_array(lst) {
2   const ret = [];
3   let idx = 0;
4   function h(curr) {
5     if (curr === null) {
6       return ret;
7     }
8     ret[idx] = head(curr);
9     idx = idx + 1;
10    return h(tail(curr));
11  }
12  return h(lst);
13 }
```

Permutations and combinations

Permutations

Description: Generates all possible permutations of elements in a list. Each permutation is a different ordering of all elements.

Parameters: ys: List - The list of elements to permute

Returns: List<List> - A list of lists, where each inner list is one permutation

Example: `permutations(list(1, 2, 3));`
// Returns: list of all 6 permutations:
// list(list(1, 2, 3), list(1, 3, 2), list(2, 1, 3),
// list(2, 3, 1), list(3, 1, 2), list(3, 2, 1))

```
1 function permutations(ys) {
2   if (ys === null) {
3     return list(null);
4   }
5   return accumulate(
6     append,
7     null,
8     map(x =>
9       map(p =>
10        pair(x, p),
11        permutations(remove(x, ys))
12      ),
13      ys
14    )
15  );
16 }
17 }
```

Combinations

Description: Generates all possible subsets (combinations) of a list, including the empty set and the full set.

Parameters: ys: List - The list of elements to find combinations of

Returns: List<List> - A list of all possible subsets

Example: `combinations(list(1, 2, 3));`
// Returns: list(list(), list(3), list(2), list(2, 3),
// list(1), list(1, 3), list(1, 2), list(1, 2, 3))

```
1 function combinations(ys) {
2   if (ys === null) {
3     return list(null);
4   }
5   const rest = combinations(tail(ys));
6   return append(
7     rest,
8     map(p => pair(head(ys), p), rest)
9   );
10 }
11 }
12 }
```

Combinations of length k

Description: Generates all combinations of exactly k elements from a list.

Parameters: ys: List - The list of elements to choose from

k: number - The number of elements in each combination

Returns: List<List> - A list of all k-length combinations

Example: `k_combinations(list(1, 2, 3, 4), 2);`
// Returns: list(list(1, 2), list(1, 3), list(1, 4),
// list(2, 3), list(2, 4), list(3, 4))

```
1 function k_combinations(ys, k) {
2   if (k === 0) {
3     return list(null);
4   } else if (ys === null) {
5     return null;
6   } else {
7     const with_head = map(
8       p => pair(head(ys), p),
9       k_combinations(tail(ys), k - 1)
10    );
11    const without_head = k_combinations(tail(ys), k);
12    return append(with_head, without_head);
13  }
14 }
```

Sorting and searching

Partition

Description: Divides a list into two sublists: elements less than or equal to the pivot, and elements greater than the pivot.

Parameters: xs: List - The list to partition

p: number - The pivot value

Returns: Pair<List, List> - A pair where the head is the "less than or equal" list and the tail is the "greater than" list

Example: `partition(list(3, 1, 4, 1, 5, 9, 2, 6), 4);`
// Returns: pair(list(3, 1, 1, 4, 2), list(5, 9, 6))

Quicksort

Description: Sorts a list in ascending order using the quicksort algorithm.

Parameters: xs: List - The list to sort

Returns: List - A new sorted list

Example: `quicksort(list(3, 1, 4, 1, 5, 9, 2, 6));`
// Returns: list(1, 1, 2, 3, 4, 5, 6, 9)

```
1 function partition(xs, p) {
2   return accumulate(
3     (curr, acc) => {
4       if (curr <= p) {
5         return pair(append(head(acc), list(curr)), tail(acc));
6       } else {
7         return pair(head(acc), append(tail(acc), list(curr)));
8       }
9     },
10    pair(list(), list()),
11    xs
12  );
13 }
14
15 function quicksort(xs) {
16
17   if (is_null(xs)) {
18     return null;
19   } else {
20     const pivot = head(xs);
21     const p = partition(tail(xs), pivot);
22     const left = head(p);
23     const right = tail(p);
24     return append(quicksort(left), append(list(pivot),
25       quicksort(right)));
26   }
27 }
```

Binary search

Description: Searches for an element in a sorted list using binary search algorithm. Returns the index if found, or -Infinity if not found. Depends on take, drop.

Parameters: lst: List - The sorted list to search in

elem: any - The element to search for

number - The index of the element, or -Infinity if not found

Example: `binary_search(list(1, 2, 3, 4, 5, 6, 7, 8, 9), 6);`
// Returns: 5

```
1 function binary_search(lst, elem) {
2   if (length(lst) === 0) {
3     return -Infinity;
4   }
5   const mid = math_floor(length(lst) / 2);
6   const val = list_ref(lst, mid);
7   if (val === elem) {
8     return mid;
9   } else if (val < elem) {
10    return mid + 1 + binary_search(drop(lst, mid + 1), elem);
11  } else {
12    return binary_search(take(lst, mid), elem);
13  }
14 }
```

Take

Description: Returns a new list containing the first n elements of a given list. If n is 0, returns null (the empty list).

Parameters: xs: List - The input list

n: number - The number of elements to take

Returns: List - A list containing the first n elements of xs

Example: `take(list(1, 2, 3, 4, 5), 3);`
// Returns: list(1, 2, 3)

```
1 function take(xs, n) {
2   return n === 0
3     ? null
4     : pair(head(xs), take(tail(xs), n - 1));
5 }
```

—

Drop

Description: Removes the first n elements from a list and returns the remainder. If n is 0, returns the original list.

Parameters: xs: List - The input list

n: number - The number of elements to skip

Returns: List - The list after removing the first n elements

Example: `drop(list(1, 2, 3, 4, 5), 2);`
// Returns: list(3, 4, 5)

```
1 function drop(xs, n) {
2   return n === 0
3     ? xs
4     : drop(tail(xs), n - 1);
5 }
```

Binary Trees

Trees are represented as list(<entry>, <left>, <right>) which is equivalent to [<entry>, [<left>, [<right>, null]]].

Insert

Description: Inserts an item into a binary search tree while maintaining the BST property. Does not insert duplicates.

Parameters: bst: Tree - The binary search tree

item: any - The item to insert

Returns: Tree - A new tree with the item inserted

Example: `insert(null, 5);`
// Returns: list(5, null, null)
`insert(list(5, null, null), 3);`
// Returns: list(5, list(3, null, null), null)

```

1 function insert(bst, item) {
2   if (bst === null) {
3     return list(item, null, null);
4   } else if (item < head(bst)) {
5     // smaller: go left
6     return list(head(bst),
7                 insert(head(tail(bst)), item),
8                 head(tail(tail(bst))));
9   } else if (item > head(bst)) {
10    // bigger: go right
11    return list(head(bst),
12               head(tail(bst)),
13               insert(head(tail(tail(bst))), item));
14   } else {
15     // equal: no duplicates. immediately return.
16     return bst;
17   }
18 }

```

Search

Description: Checks if an element exists in a binary search tree. Assumes no duplicates.

Parameters: bst: Tree - The binary search tree to search

Returns: target: any - The value to search for

boolean - True if the element is found, false otherwise

Example: `const tree = list(5, list(3, null, null), list(7, null, null));`

`search(tree, 3);` // Returns: true

`search(tree, 10);` // Returns: false

```

1 function search(bst, target) {
2   if (bst === null) {
3     return false;
4   } else if (head(bst) === target) {
5     return true;
6   } else if (target < head(bst)) {
7     // smaller, search left
8     return search(head(tail(bst)), target);
9   } else {
10    // larger, search right
11    return search(head(tail(tail(bst))), target);
12  }
13 }

```

Traverse

Description: Traverses a binary tree and returns a list of values in the specified order:

- **Pre-order:** root, left, right

- **In-order:** left, root, right (produces sorted list for BST)

- **Post-order:** left, right, root

Parameters: bst: Tree - The binary tree to traverse

Returns: List - A list of tree values in the specified traversal order

Example: `// Tree: 5`
`// /\`
`// 3 7`
`const tree = list(5, list(3, null, null), list(7, null, null));`
`preorder(tree);` // Returns: list(5, 3, 7)
`inorder(tree);` // Returns: list(3, 5, 7)
`postorder(tree);` // Returns: list(3, 7, 5)

```

1 function append3(a, b, c) {
2   return append(
3     a, append(b, c)
4   );
5 }
6
7 function preorder(bst) {
8   if (bst === null) {
9     return null;
10  } else {
11    return append3(
12      list(head(bst)),
13      preorder(head(tail(bst))),
14      preorder(head(tail(tail(bst))))
15    );
16  }
17 }
18
19 function inorder(bst) {
20   if (bst === null) {
21     return null;
22   } else {
23     return append3(
24       inorder(head(tail(bst))),
25       list(head(bst)),
26       inorder(head(tail(tail(bst))))
27     );
28   }
29 }
30
31 function postorder(bst) {
32   if (bst === null) {
33     return null;
34   } else {
35     return append3(
36       postorder(head(tail(bst))),
37       postorder(head(tail(tail(bst)))),
38       list(head(bst))
39     );
40   }
41 }

```

Delete

Description: Removes a target value from a binary search tree while maintaining the BST property. Uses the

inorder successor strategy for nodes with two children.

Parameters: bst: Tree - The binary search tree

target: any - The value to delete

Returns: Tree - A new tree with the target removed

Example: `// Tree: 5`
`// /\`
`// 3 7`
`const tree = list(5, list(3, null, null), list(7, null, null));`
`delete(tree, 3);` // Returns list(5, null, list(7, null, null))

Find Min

Description: Finds the smallest (leftmost) value in a binary search tree.

Parameters: bst: Tree - The binary search tree

Returns: any - The minimum value in the tree

Example: `// Tree: 5`
`// /\`
`// 3 7`
`const tree = list(5, list(3, null, null), list(7, null, null));`
`find_min(tree);` // Returns: 3

```

1 function delete(bst, target) {
2   if (bst === null) {
3     return bst; // nothing to delete
4   } else if (target < head(bst)) {
5     // smaller: go left
6     return list(
7       head(bst),
8       delete(head(tail(bst)), target),
9       head(tail(tail(bst)))
10    );
11   } else if (target > head(bst)) {
12     // bigger: go right
13     return list(
14       head(bst),
15       head(tail(bst)),
16       delete(head(tail(tail(bst))), target)
17    );
18   } else {
19     // found node to delete
20     if (head(tail(bst)) === null) {
21       return head(tail(tail(bst))); // only right child or none
22     } else if (head(tail(tail(bst))) === null) {
23       return head(tail(bst)); // only left child
24     } else {
25       // two children: replace with inorder successor
26       // successor is leftmost number of the right branch
27       const successor = find_min(head(tail(tail(bst))));
28       return list(
29         successor,
30         head(tail(bst)),
31         // remove the smallest number in the right branch
32         delete(head(tail(tail(bst))), successor)
33       );
34     }
35   }
36 }
37
38 // find the smallest number in tree
39 function find_min(bst) {
40   if (head(tail(bst)) === null) {
41     return head(bst);
42   } else {
43     return find_min(head(tail(bst)));
44   }
45 }

```

Invert

Description: Inverts (mirrors) a binary tree by swapping the left and right subtrees of every node.

Parameters: bst: Tree - The binary tree to invert

Returns: Tree - A new tree that is the mirror image of the input

Example: `// Original: 5`
`// /\`
`// 3 7`
`const tree = list(5, list(3, null, null), list(7, null, null));`
`invert(tree);` // Returns list(5, list(7, null, null), list(3, null, null))

```

1 function invert(bst) {
2   if (bst === null) {
3     return bst;
4   } else {
5     return list(
6       head(bst),
7       invert(head(tail(tail(bst)))), // swap sides
8       invert(head(tail(bst)))
9     );
10  }
11 }

```

Flatten

Description: Flattens a nested tree structure into a single-level list by recursively processing all sublists.

Parameters: tree: Tree - The tree structure to flatten

Returns: List - A flat list containing all leaf values

Example: `flatten_tree(list(1, list(2, 3), list(4, list(5, 6))));`

// Returns: list(1, 2, 3, 4, 5, 6)

```

1 function flatten_tree(tree) {
2   return accumulate(
3     (x, y) => is_list(x)
4       ? append(flatten(x), y)
5       : pair(x, y),
6     null,
7     tree);
8 }

```

Map

Description: Applies a function to every leaf value in a tree structure, preserving the tree's shape.

Parameters: f: Function - The function to apply to each leaf value

tree: Tree - The tree structure to map over

Returns: Tree - A new tree with the function applied to all leaves

Example: `map_tree(x => x * 2, list(1, list(2, 3), 4));`

// Returns: list(2, list(4, 6), 8)

```

1 function map_tree(f, tree) {
2   return map(
3     x => is_list(x)
4       ? map_tree(f, x)
5       : f(x),
6     tree);
7 }

```

Accumulate/Reduce

Description: Reduces a tree structure to a single value by applying a binary function to all leaf values with an initial accumulator.

Parameters: f: Function - Binary function (element, accumulator) => new_accumulator

init: any - Initial accumulator value

tree: Tree - The tree structure to reduce

Returns: any - The final accumulated value

Example: `accumulate_tree((x, y) => x + y, 0, list(1, list(2, 3), 4));`

// Returns: 10 (sum of all values)

```

1 function accumulate_tree(f, init, tree) {
2   return accumulate(
3     (x, y) => is_list(x)
4       ? accumulate_tree(f, y, x)
5       : f(x, y),
6     init,
7     tree);
8 }

```